

Digital Signal Processing Lab (CSE 3210)

Experiment 1: Introduction to Python for DSP Applications

1. Objectives

- To become familiar with Python programming basics required for DSP.
- To practice variable handling, input/output, and data types.
- To understand NumPy arrays and basic array operations.
- To visualize data using Matplotlib (basic line plots and stem plots).
- To build a foundation for implementing DSP algorithms in later labs.

2. Course Outcome (CO)

- **CO1:** Apply mathematical knowledge to implement and analyze various signal processing systems, enhancing understanding of their principles and applications.

3. Program Outcome (PO) Mapping

- **PO2:** Problem Analysis — Students practice analyzing computational problems (data manipulation, plotting) and apply mathematical knowledge to solve them.

Delivery methods: Lecture, Tutorial, Lab Manual

Assessment Tools: Lab Performance, Lab Report.

4. Knowledge Profile (KPA) Mapping

- **K1:** Apply mathematics and numerical analysis (array operations, numeric types).
- **K3:** Apply engineering fundamentals (use of programming tools to represent and manipulate signals).

5. Complex Engineering Problem (CEP) Mapping

- **P1:** Depth of knowledge required — Translating math into code.

6. Background Theory

This experiment does not require DSP theory. Instead, it builds Python programming fundamentals which are essential tools for digital signal processing. In later labs, these skills will be used to implement signal operations, transformations, and filtering.

Topics covered:

- Printing, variables, and types.
- Input and type conversion.
- Strings and slicing.
- NumPy arrays and matrix operations.
- Random numbers and identity matrices.
- Plotting with Matplotlib.

7. Lab Tasks

Task 1: Python Basics

- Declare variables of different types (int, float, string, bool) and display their types.
- Take input from the user and convert it to integer/float.

Task 2: Strings and Concatenation

- Access characters and substrings using indexing and slicing.
- Concatenate two strings with the + operator.

Task 3: NumPy Arrays

- Create 1D and 2D arrays using `np.array()`.
- Find their shape, dimension, and datatype.
- Use functions like `zeros()`, `ones()`, `full()`, `identity()`.

Task 4: Array Operations

- Perform elementwise addition, subtraction, multiplication.
- Demonstrate broadcasting (e.g., multiply array by scalar).

Task 5: Random Numbers

- Generate random numbers using `np.random.rand()` and plot a histogram.

Task 6: Plotting with Matplotlib

- Plot $y=2x$ for $x=1,3,5,\dots,9$.
- Plot multiple functions on the same figure (e.g., $y=x$, $y=x^2$).
- Use `plt.stem()` to show discrete points.

8. Sample Code Snippets

<https://drive.google.com/file/d/1YDnHzgiQi2V4egy9JOu4sC5SxWU0nIhU/view?usp=sharing>

9. Expected Outcomes

- Students will understand how to represent data using arrays.
- Students will be able to perform mathematical computations programmatically.
- Students will learn basic plotting tools, preparing them for DSP visualization.

10. Lab Report:

a) Find out the n 'th fibonacci number.

b) **Student ID as Data:**

- Take each digit of your student ID and store it in a NumPy array.
- Compute:
 - Sum, mean, variance.
 - A new array where each element is squared.
- Plot the digits as a stem plot.

c) Image Pixel Array

- Use NumPy to create a 5×5 matrix with random integers between 0 and 255.
- Display it using `plt.imshow()`.

d) Compare Growth Rates

- Plot the following functions together for $x=0 \dots 10$:
 - Linear ($y=x$), Quadratic ($y=x^2$), Exponential ($y=2^x$).
- Label each curve.

e) Encode Your Name as Signal

- Convert each character of your name into ASCII values.
- Store as a NumPy array.
- Plot it with `stem()`.